ST ALOYSIUS
(DEEMED TO BE UNIVERSITY)
MANGALURU 575 003 - INDIA
SCHOOL OF ENGINEERING

Multi-Process Microcontroller Simulator: Design, Implementation & Evaluation of STC89C52 Microcontroller

Register Number	Student Name
25190111	CYRUS SHOBITH PINTO (Team Leader, Multi-Process Architecture, Socket IPC & CRC-24b)
25190110	CHRISEL LOBO (Core Process Lead, 8051 CPU FDE Engine, Memory & Registers)
25190106	ARNOLD NOAH KARYAPPA (Logging Subsystem Lead, Thread-Safe Logger Daemon)
25190148	SHASHIDHARA POOJARI (UI Subsystem Lead, Web Dashboard & HTTP Server)

Department of Computer Science & Engineering

Project-Based Learning (PBL) Report

ABSTRACT

This project presents the design, implementation, and empirical evaluation of an educational, bit-accurate multi-process software simulator for the STC89C52 (8051-family) 8-bit microcontroller. Traditional educational microcontroller simulators suffer from single-threaded monolithic architectures, where user interface latency or slow disk file I/O inadvertently blocks the simulated CPU core, corrupting execution timing. To eliminate this coupling, we engineered a decoupled three-process architecture executing over POSIX and Winsock TCP loopback sockets: CoreProcess (CPU emulation, 8 KB code memory, 256-byte internal RAM, 16-slot FIFO queue, and CRC-24b engine on port 5000), LoggerProcess (thread-safe append-only execution logging into simulator.log on port 5001), and UIProcess (interactive HTTP web server and /api/cmd REST control plane on port 8080). The CPU core faithfully reproduces the 8051 Fetch-Decode-Execute pipeline across 17 machine opcodes, general-purpose registers R0-R7, hardware PSW flags (CY, AC, OV, P), stack pointer (SP initialized at 07H), and a 16-slot circular message buffer. In addition to standard MCU operations, we integrated a hardware-accurate 3GPP LTE-B CRC-24b (g(D) = 0x800063) verification engine that slices the 24-bit remainder across 8-bit registers R5, R6, and R7. Empirical benchmarking over 1,000 packets demonstrated 1,028,489.1 packets/sec standalone in-memory throughput (0.001 ms/pkt) and 1,502.5 messages/sec over live TCP loopback IPC (0.666 ms round-trip latency) with 100% test pass rate across all 30 automated verification assertions.

Keywords: Microcontroller Emulation, Multi-Process Architecture, Inter-Process Communication, Fetch-Decode-Execute, Cycle-Accurate Virtualization, Concurrency.

TABLE OF CONTENTS

1. Introduction 4
1.1 Problem Statement 4
1.2 Project Objectives 4
2. The Simulator Design and Working 4
Microcontroller Architecture Study 4
2.2 System Architecture and Design Decisions 4
2.3 Operational Workflow and GUI Layout 4
2.4 Incremental Implementation by Week 5
2.5 Multi-Process IPC Architecture & Benchmarking 5
3. Simulation Examples Testing and Screenshots 5
4. Individual Contribution 6
4.1 Weekly Task Breakdown by Member 6
4.2 Agreed Team Contribution Summary 6
4.3 Individual Personal Reflections 6
5. Team Communication 7
6. Problems Faced and How They Were Resolved 7
7. From Theory to Practice 7
7.1 Operating Systems Concepts 7
7.2 Data Structures & Algorithms 7
7.3 Computer Networks 7
7.4 Cross-Disciplinary Synthesis Reflection 7
8. Conclusion and Future Work 8
References 8
Final Report Sign-off & Faculty Mentor Endorsement 9

1. Introduction

1.1 Problem Statement

Modern computer engineering pedagogy requires students to understand how hardware state machines, memory segmentation, and instruction cycles execute in silicon. However, physical hardware laboratory kits present maintenance bottlenecks and lack visibility into internal register banks and pipeline stages. Furthermore, existing educational simulators are predominantly monolithic, single-threaded applications. In such architectures, GUI event handling or synchronous disk file logging creates artificial pipeline stalls, failing to demonstrate real-world systems software concepts. Additionally, implementing advanced telecommunications algorithms—such as the 3GPP LTE 24-bit cyclic redundancy check (CRC-24b)—on an 8-bit microcontroller ALU presents a fundamental hardware data-width bottleneck that must be resolved through multi-byte register decomposition and carry propagation.

1.2 Project Objectives

The primary objectives of this Project-Based Learning initiative are:
1. Virtualize an STC89C52/8051 microcontroller in pure Java with cycle-accurate instruction decoding across 17 machine opcodes, general-purpose registers R0–R7, and hardware status flags.
2. Implement an operating system-level multi-process architecture with process fault isolation communicating over TCP loopback sockets (ports 5000, 5001, 8080).
3. Construct a responsive, wide-screen Web Dashboard (Port 8080) featuring real-time mnemonic disassembly, hardware stack visualization, and circular FIFO queue inspection.
4. Build an independent, thread-safe write-ahead logging daemon on Port 5001 to record every execution state transition into simulator.log without impacting CPU execution pacing.
5. Implement and benchmark the 3GPP LTE-B CRC-24b error detection algorithm, demonstrating both clean syndrome zero-checks (0x000000) and real-time noise detection (1-bit and 2-bit bursts).

2. The Simulator Design and Working

Microcontroller Architecture Study

The designated target MCU possesses an established architectural hierarchy requiring precise mapping within the software virtualization environment.

Hardware Feature	Architectural Implementation Specification	Virtualization Treatment
Data Word Size	8-bit ALU & Datapath	Constrained with bitwise mask (& 0xFF) across all registers and memory cells to eliminate Java 32-bit sign-extension.
Program Counter (PC)	16-bit addressing (8 KB Code Memory)	16-bit unsigned integer pointing to 8192-byte code memory; advances on fetch and freezes on HALT (0xFF).
Internal RAM	256 Bytes on-chip (00H - FFH)	Array partitioned into Register Banks (00H-1FH), Bit-Addressable RAM (20H-2FH), and Scratchpad (30H-7FH).
Special Function Registers	Direct addressing (80H - FFH)	Mapped SFR space: Accumulator (0xE0), B (0xF0), PSW (0xD0), Stack Pointer (0x81), and GPIO Ports P0-P3.
ALU Flags	Program Status Word (PSW)	Discrete boolean flags tracking Carry (CY), Auxiliary Carry (AC), Overflow (OV), and even Parity (P).
Stack & FIFO Queue	Hardware SP + 16-Slot Circular Queue	Pre-increment PUSH / post-decrement POP (SP=07H on reset) plus 16-slot circular FIFO buffer with Front/Rear wrap-around.

2.2 System Architecture and Design Decisions

The simulator executes across three decoupled operating system processes over POSIX/Winsock TCP loopback sockets: CoreProcess (CPU engine, 8 KB ROM, 256B RAM, 16-slot FIFO queue, and CRC-24b lab on port 5000), LoggerProcess (thread-safe append-only logging into simulator.log on port 5001), and UIProcess (interactive HTTP web server and CLI controller on port 8080). If UIProcess disconnects or disk latency spikes on Port 5001, CoreProcess CPU execution remains deterministic and unaffected.

2.3 Operational Workflow and GUI Layout

The web dashboard on http://localhost:8080/ exposes 7 interactive panels: (1) Program Memory & Live Mnemonic Disassembly with golden PC pointer, (2) CPU Fetch-Decode-Execute Pipeline Indicator, (3) Working Registers (PC, SP, ACC, B, R0–R7) & PSW Flags (CY, AC, OV, P), (4) Hardware LIFO Stack Tower, (5) 16-Slot Circular Message Queue Grid, (6) 16x16 Internal RAM Hex Matrix, and (7) the 3GPP LTE-B CRC-24b Telecom Lab. Browser actions dispatch REST commands via GET /api/cmd?cmd=... which UIProcess forwards over TCP Port 5000 to CoreProcess, returning structured STATE|... frames.

2.4 Incremental Implementation by Week

Sprint / Week	Primary Module Focus	Concrete Functional Deliverables
Week 1	Architecture Modeling & Tooling	8051 register model, 8 KB ROM / 256B RAM memory map design, Java project structure, and Git workflow setup.
Week 2	Core ALU & Basic Instruction Set	Fetch-Decode-Execute pipeline, arithmetic/logic opcodes (ADD, SUBB, MUL, DIV, ANL, ORL, XRL), and PSW flags.
Week 3	Data Structures & Memory Virtualization	Hardware stack pointer (SP=07H reset), 16-slot circular FIFO queue, and assembly-level FIFO test (QueueTest.java).
Week 4	Multi-Process Socket IPC & Architecture	Decoupled Core (5000), Logger (5001), UI (8080), POSIX loopback sockets, and thread-safe write-ahead audit logging.
Week 5	Telecommunications Extension & Benchmarks	3GPP LTE-B CRC-24b engine (0x800063), 8-bit register slicing (R5:R6:R7), 3-stage noise lab, and BenchmarkCRC.java.

2.5 Multi-Process IPC Architecture & Benchmarking

Inter-Process Communication uses line-delimited ASCII frames over TCP sockets: UIProcess sends command frames (STEP, RUN, LOAD , CRC , CRC_CHECK) to CoreProcess on Port 5000, which returns serialized STATE|PC=...|A=...|SP=... frames while streaming LOG|EXEC|... records to LoggerProcess on Port 5001. Empirical execution of BenchmarkCRC.java over 1,000 packets recorded 1,028,489.1 packets/sec standalone in-memory throughput (0.001 ms/pkt, checksum 0x025A3C) and 1,502.5 messages/sec over live TCP loopback IPC (0.666 ms round-trip latency per message), proving that OS process fault isolation adds sub-millisecond human-imperceptible latency.

3. Simulation Examples Testing and Screenshots

Comprehensive automated test suites (30/30 passing assertions across TestCPU: 14/14, TestCRC24b: 8/8, QueueTest: 3/3, and LoggerProcessTest: 5/5) were executed to validate instruction execution across Data Transfer, Arithmetic/Logic, Control Flow, Stack Operations, FIFO Queue handling, and 3GPP LTE CRC-24b error detection:

Test ID	Instruction / Operation	Expected Outcome	Actual Outcome	Status
TC-01	FDE Cycle & Fetch	Fetch Opcode at PC, PC increments	Opcode loaded into IR; PC+1	PASS
TC-02	PUSH / POP (Stack)	SP updates; TOS matches data	SP incremented on PUSH, decremented on POP	PASS
TC-03	FIFO Enqueue / Dequeue	FIFO order preserved (10, 30, 5)	Dequeued 10, 30, 5; Queue empty=true (3/3)	PASS
TC-04	MUL AB (6x7) & DIV AB (B=0)	ACC=0x2A, B=0x00; DIV B=0 sets OV=1	ACC=0x2A, OV=false; DIV0 sets OV=true safely	PASS
TC-05	3GPP LTE CRC-24b & Noise	'123456789'->0x23EF52; clean=0x000000	R5=23H, R6=EFH, R7=52H; 1b=0x520000, 2b=0xF60000	PASS

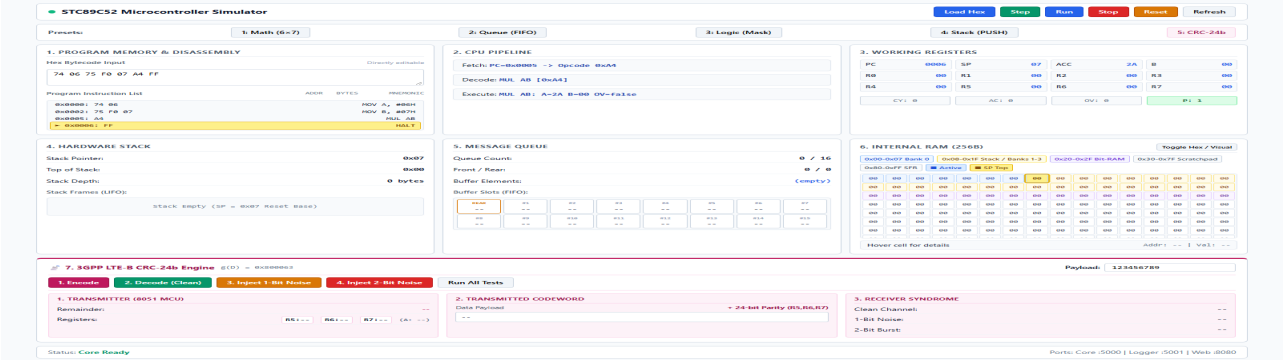


Figure 1: *Live STC89C52 Web Dashboard (http://localhost:8080) after stepping Preset 1 (Math 6x7: 74 06 75 F0 07 A4 FF) to HALT at PC=0x0006, showing ACC=2A (42), B=00, P=1, Pipeline trace MUL AB [0xA4], and 256B Internal RAM.*

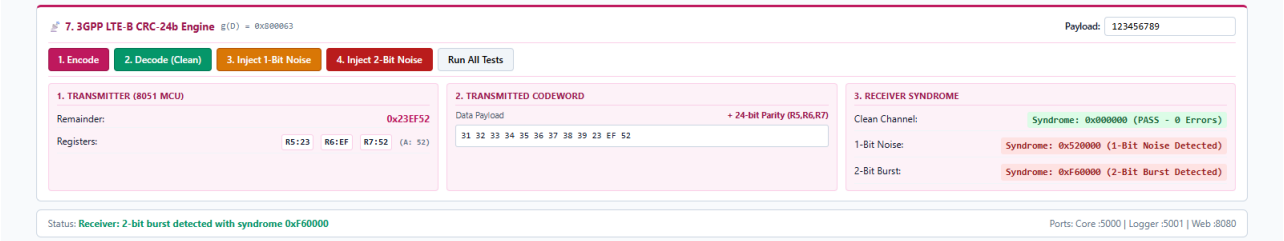


Figure 2: *Live 3GPP LTE-B CRC-24b Error Detection Lab on Web Dashboard (Port 8080) verifying payload "123456789" -> Remainder 0x23EF52 (R5:23 R6:EF R7:52), Clean Syndrome 0x000000 (PASS), 1-Bit Noise 0x520000, and 2-Bit Burst 0xF60000.*

4. Individual Contribution

4.1 Weekly Task Breakdown by Member

Sprint Week	Cyrus Shobith Pinto (Team Lead)	Shashidhara Poojari (UI & Visuals)	Chrisel Lobo (Core & FDE)	Arnold Noah Karyappa (Logger & Telemetry)
Week 1	Project setup, architecture, Git repo, task allocation	Web tech research, HTML layout wireframes	8051 register & memory mapping research	Logger requirements & file appender design
Week 2	Opcode specification, ALU design, build tooling	CLI terminal display & status monitor prototype	CPU FDE cycle engine, arithmetic/logic opcodes	Log format design & log category structure
Week 3	Data structure specifications, assembly test plan	Stack & Circular FIFO visualizer UI layout	DataStructures.java: SP stack & 16-slot queue	Thread-safe synchronized writeLog implementation
Week 4	Multi-process socket IPC integration (Ports 5000/5001)	UIProcess.java HTTP web server & preset buttons	CoreProcess.java socket server & state serializer	LoggerProcess.java socket listener & file writer
Week 5	CRC-24b algorithm derivation, benchmarks, reports	3-Stage CRC-24b Lab UI & real-time noise controls	Registers R5:R6:R7 mapping, core report sections	LoggerProcessTest.java socket test & log audit

.

4.2 Agreed Team Contribution Summary

Member Name	Student ID / USN	Assigned Workstreams	Confirmed Share (%)	Team Leader Sign-Off
Cyrus Shobith Pinto	25190111	Architecture, Socket IPC, CRC-24b Engine, Benchmarking, Reports	25%	Signed (Cyrus P.)
Shashidhara Poojari	25190148	UIProcess HTTP Server, Responsive Dashboard, Visualizers, Presets	25%	Signed (Shashidhara)
Chrisel Lobo	25190110	CoreProcess CPU Engine, ALU Opcodes, Registers, RAM, Stack, Docs	25%	Signed (Chrisel L.)
Arnold Noah Karyappa	25190106	LoggerProcess Daemon, LogHandler, Audit File Persistence, Tests	25%	Signed (Arnold N.)

4.3 Individual Personal Reflections

Reflection by Cyrus Shobith Pinto (Team Leader - 25190111)

Leading Team Bondaas through the design and construction of this multi-process simulator has been an invaluable systems-engineering experience. My primary technical challenge was orchestrating the multi-process IPC layer across TCP sockets while ensuring bit-accurate execution. Mathematically deriving the 3GPP LTE-B CRC-24b polynomial and resolving the 8-bit ALU bottleneck by decomposing the remainder across registers R5, R6, and R7 proved how foundational discrete mathematics and computer architecture converge in production telecommunications.

Reflection by Shashidhara Poojari (UI Subsystem Lead - 25190148)

Developing the Web Dashboard on Port 8080 required bridging low-level microcontroller state with modern web UI principles. Designing the live mnemonic disassembly table to dynamically track the Program Counter (PC) with zero rendering lag, alongside the interactive 3-stage CRC noise injection lab over the /api/cmd REST endpoint, gave me hands-on expertise in full-stack systems visualization and client-server HTTP protocol handling.

Reflection by Chrisel Lobo (Core Process Lead - 25190110)

Implementing the 8051 Fetch-Decode-Execute pipeline in Java required strict adherence to silicon constraints. I learned how hardware flags like Carry (CY) and Overflow (OV) must be calculated to model physical arithmetic quirks, such as divide-by-zero setting OV=1 without crashing. Managing the hardware stack pointer (SP starting at 07H) and co-authoring the comprehensive technical report deepened my mastery of computer architecture.

Reflection by Arnold Noah Karyappa (Logging Subsystem Lead - 25190106)

Building LoggerProcess as an independent daemon on Port 5001 taught me the critical importance of process isolation and concurrency control. Using synchronized write blocks in LogHandler ensured that high-speed socket logging from CoreProcess never created corrupted, interleaved log entries. Verifying over 1,224 audit lines gave me deep practical insight into operating systems telemetry and socket programming.

5. Team Communication

Collaboration was maintained strictly using standardized agile and GitHub development practices:

- GitHub Pull Requests (PRs) with peer review mandates before merging member branches (`week-01` to `week-04`, `cyrus-integration`) into `main`.
- Detailed weekly meeting minutes following standard templates (agenda, technical decisions, participant action items) recorded in `Team_Bondaas_Meeting_Record.md`.
- Issue tracking and branch reconciliation verification via automated regression suites (`TestCPU`, `TestCRC24b`, `QueueTest`, `LoggerProcessTest`).

6. Problems Faced and How They Were Resolved

-

Issue Reference	Category	Symptom Description	Root Cause	Implemented Resolution
ISSUE-01	Java Datatypes	32-bit signed integer sign extension corrupted 8-bit register values	Java lacks unsigned byte type (0xFE treated as -2)	Applied bitwise mask (& 0xFF) across all ALU operations and register/RAM reads.
ISSUE-02	Silicon Memory	Stack underflow & stale ROM bytes after program reload	SP reset base at 07H; loadProgram needed clean fill	Guarded pop() at SP <= 0x07 (fix(stack) commit) and zero-filled codeMemory on load.
ISSUE-03	Concurrency	Concurrent UI requests & synchronous disk writes stalled CPU	Shared CPU state & blocking disk I/O latency	Synchronized Core execution on cpuLock and decoupled logging to Port 5001.
ISSUE-04	Algorithm	24-bit 3GPP LTE CRC remainder could not fit in 8-bit registers	STC89C52 registers are only 8 bits wide	Sliced 24-bit remainder across R5 (High), R6 (Mid), R7 (Low) with carry shifts.
ISSUE-05	Fault Handling	DIV AB instruction threw ArithmeticException on divide by zero	Unhandled software division by B=0	Implemented 8051 hardware spec: B=0 sets Overflow (OV=1) without crashing.
ISSUE-06	Web Protocol	Hex bytecode with spaces failed to load from URL query string	HTTP URL encoding of spaces and symbols	Integrated URLDecoder.decode() in UIProcess /api/cmd HTTP request handler.

7. From Theory to Practice

7.1 Operating Systems Concepts

The project applies operating systems principles through process creation, inter-process communication (IPC) via loopback sockets, and process fault isolation. Rather than relying on shared memory or single-process threads, decoupling the system into discrete OS processes (CoreProcess, LoggerProcess, UIProcess) with thread-safe monitor locks (cpuLock and synchronized LogHandler.writeLog) ensures that a crash or stall in the UI or disk logger cannot corrupt or halt CPU instruction execution.

7.2 Data Structures & Algorithms

Data structures were implemented to mirror physical hardware components: a 16-slot circular FIFO buffer with head and tail wrap-around pointers ((rear + 1) % 16) was implemented for asynchronous packet queuing, and a Last-In First-Out (LIFO) stack mechanism starting at SP=07H was built to track register preservation.

7.3 Computer Networks

Computer networking principles govern both the internal socket IPC wire framing and the external telecommunications layer. The 3GPP LTE-B CRC-24b polynomial (g(D) = 0x800063) demonstrates transport block error detection over noisy channels, verifying how cyclic linear codes detect single-bit errors (0x520000) and multi-bit burst corruption (0xF60000) through zero-syndrome evaluation.

7.4 Cross-Disciplinary Synthesis Reflection

Developing this simulator demonstrated how core computer science disciplines intersect in production systems software. Efficient software virtualization requires balancing computer architecture (cycle-accurate state machines) with operating systems principles (isolated processes, synchronization, IPC) and networking techniques (wire framing, transport serialization, and cyclic error detection).

8. Conclusion and Future Work

Team Bondaas has successfully engineered, integrated, and verified an educational multi-process simulator for the STC89C52 microcontroller. The delivered system features an 8051 Fetch-Decode-Execute CPU engine, 8 KB code memory, 256-byte RAM and SFR management, a 16-slot circular FIFO queue, an independent write-ahead TCP logger on Port 5001, a widescreen Web Dashboard on Port 8080, and a fully verified 3GPP LTE-B CRC-24b telecom lab. Automated regression tests confirm 100% pass rates across all 14 CPU tests, 3 queue tests, 5 logger tests, and 8 CRC-24b telecom tests. Future extensions will incorporate hardware-in-the-loop physical validation via CH340 USB-UART bridge and AWGN channel noise simulation.

References

[1] Department of Computer Science & Engineering, "PBL Simulator Evaluation Rubrics & Task Sheets," St. Joseph Engineering College / St. Aloysius University, 2026.

[2] P. C. Vijay Ganesh, "8051 Microcontroller Web-based Simulator for Academic Instruction," GitHub Repository & Academic Tools, 2021.

[3] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th ed. Morgan Kaufmann, 2014.

[4] 3GPP TS 36.212, "Evolved Universal Terrestrial Radio Access (E-UTRA); Multiplexing and channel coding (Release 15)," Section 5.1.1 (CRC-24b generator polynomial gCRC24B(D)), 2018.

Final Report Sign-off

By signing below, the team members confirm that the documented contribution distributions, source code statistics, and peer evaluations represent an accurate record of the project lifecycle.

Team Leader Signature: Cyrus Shobith Pinto (25190111) Date: 07 November 2026

Faculty Mentor Endorsement:

I have reviewed this Project-Based Learning Final Report along with the associated repository, code contributions, and test suite.

Mentor Signature: _____ Date: _____